

CEE 360 Final Project

Emyline Kirda, Jeff Gao, Robert Skrentny, Zoeya Khan

5/6/2026

- Using the standard equation relating members, reaction forces, and joints ($m + r = 2j$), the bridge structure is statically determinate and there is enough information to solve for all the unknowns. The structure is also stable as there are horizontal and vertical reaction forces and the lines of action from the two upward reactions are separate so it cannot rotate.

$$m = 17$$

$$j = 10$$

$$r = 3$$

$$m + r = 2j$$

$$17 + 3 = 2(10)$$

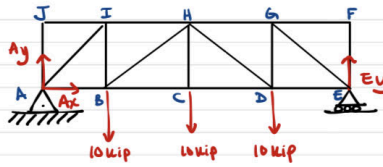
$$20 = 20$$

- Structural Analysis Using the Principle of Virtual Work

$$Deflection = \sum \frac{nNL}{EA}$$

$$Mid Span Deflection = \frac{33972.35 \text{ kip}\cdot\text{in}}{(29,000 \text{ ksi})(26.2 \text{ in}^2)} = 0.04471222705 \text{ in (downward)}$$

Member	n (kip)	N (kip)	L (in)	nNL
Ax	0	0	-	-
Ay	0.5	15	-	-
Ey	0.5	15	-	-
JA	0	0	240	0.00
IJ	0	0	240	0.00
IA	-0.7071067812	-21.21	339.36	5089.63
BA	0.5	15	240	1800.00
IH	-0.5	-15	240	1800.00
IB	0.5	15	240	1800.00
HB	-0.7071067812	-7.07	339.36	1696.54
BC	1	20	240	4800.00
CD	1	20	240	4800.00
HC	0	10	240	0.00
HD	-0.7071067812	-7.07	339.36	1696.54
HG	-0.5	-15	240	1800.00
ED	0.5	15	240	1800.00
GD	0.5	15	240	1800.00
EG	-0.7071067812	-21.21	339.36	5089.63
EF	0	0	240	0.00
GF	0	0	240	0.00
				33972.35



Virtual Work equation

$$1 \cdot \Delta = \sum u \cdot \Delta L = \sum \frac{nNL}{EA}$$

n = virtual member
 N = real member

REAL EQUILIBRIUM SYSTEM:

$$\begin{aligned} \sum F_y &= A_y + B_y - 10k - 10k - 10k = 0 \\ \sum F_y &= A_y + B_y - 30k = 0 \\ \sum F_x &= A_x = 0 \text{ kip} \\ + \sum M_A &= E_y(4 \times 20ft) - 10k(20ft) - 10k(40ft) - 10k(60ft) = 0 \\ \frac{80B_y}{80} &= \frac{1200 \text{ k-ft}}{80} \\ B_y &= 15 \text{ kip} \\ A_y &= 30k - B_y \\ A_y &= 30k - 15k \\ A_y &= 15 \text{ kip} \end{aligned}$$

REAL MEMBER FORCES: (Method of Joints)

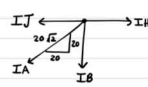
JOINT J:

$$\begin{aligned} \sum F_y &= JI = 0 \text{ kip} \\ \sum F_x &= IJ = 0 \text{ kip} \end{aligned}$$

JOINT A:

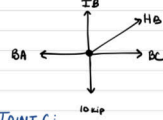
$$\begin{aligned} \sum F_y &= IA \left(\frac{20}{\sqrt{20^2 + 15^2}} \right) + A_y + JI = 0 \\ \left(\frac{1}{\sqrt{2}} \right) (IA) &= -A_y \\ IA &= (-15) \left(\frac{1}{\sqrt{2}} \right) \\ IA &= -21.21 \text{ kip} \quad (\text{compression}) \\ \sum F_x &= BA + IA \left(\frac{15}{\sqrt{2}} \right) = 0 \\ BA &= -IA \left(\frac{1}{\sqrt{2}} \right) \\ BA &= 15 \text{ kip} \quad (\text{tension}) \end{aligned}$$

JOINT I:



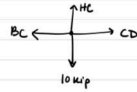
$$\begin{aligned}\sum F_x &= I_H - I_J - I_A\left(\frac{1}{\sqrt{2}}\right) = 0 \\ I_H &= I_A\left(\frac{1}{\sqrt{2}}\right) \\ I_H &= (-21.21 \text{ kip})\left(\frac{1}{\sqrt{2}}\right) \\ I_H &= -15 \text{ kip} \quad (\text{compression}) \\ \sum F_y &= -I_A\left(\frac{1}{\sqrt{2}}\right) - I_B = 0 \\ -I_A\left(\frac{1}{\sqrt{2}}\right) &= I_B \\ I_B &= 15 \text{ kip} \quad (\text{tension})\end{aligned}$$

JOINT B:



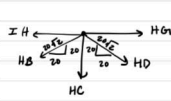
$$\begin{aligned}\sum F_y &= I_B + H_B\left(\frac{1}{\sqrt{2}}\right) - 10 \text{ kip} = 0 \\ H_B &= \sqrt{2}(10 \text{ kip} - 15 \text{ kip}) \\ H_B &= -7.07 \text{ kip} \quad (\text{compression}) \\ \sum F_x &= -B_A + B_C + H_B\left(\frac{1}{\sqrt{2}}\right) = 0 \\ B_C &= B_A - H_B\left(\frac{1}{\sqrt{2}}\right) \\ B_C &= 15 \text{ kip} - (-7.07)\left(\frac{1}{\sqrt{2}}\right) \\ B_C &= 20 \text{ kip} \quad (\text{tension})\end{aligned}$$

JOINT C:



$$\begin{aligned}\sum F_x &= C_D - B_C = 0 \\ C_D &= 20 \text{ kip} \quad (\text{tension}) \\ \sum F_y &= H_C - 10 \text{ kip} = 0 \\ H_C &= 10 \text{ kip} \quad (\text{tension})\end{aligned}$$

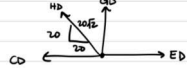
JOINT H:



$$\begin{aligned}\sum F_y &= -H_C - H_D\left(\frac{1}{\sqrt{2}}\right) - H_B\left(\frac{1}{\sqrt{2}}\right) = 0 \\ \sqrt{2}[-H_C - H_D\left(\frac{1}{\sqrt{2}}\right)] &= H_D \\ \left(\frac{1}{\sqrt{2}}\right)(-10 \text{ kip}) + 7.07 \text{ kip} &= H_D \\ H_D &= -7.07 \text{ kip} \quad (\text{compression})\end{aligned}$$

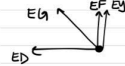
$$\begin{aligned}\sum F_x &= H_G - I_H + H_D\left(\frac{1}{\sqrt{2}}\right) - H_B\left(\frac{1}{\sqrt{2}}\right) = 0 \\ H_G &= I_H - H_D\left(\frac{1}{\sqrt{2}}\right) + H_B\left(\frac{1}{\sqrt{2}}\right) \\ H_G &= (-15 \text{ kip}) + 7.07 \text{ kip}\left(\frac{1}{\sqrt{2}}\right) - 7.07 \text{ kip}\left(\frac{1}{\sqrt{2}}\right) \\ H_G &= -15 \text{ kip} \quad (\text{compression})\end{aligned}$$

JOINT D:



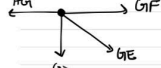
$$\begin{aligned}\sum F_x &= E_D - C_D - H_D\left(\frac{1}{\sqrt{2}}\right) = 0 \\ E_D &= 20 \text{ kip} + (-7.07 \text{ kip})\left(\frac{1}{\sqrt{2}}\right) \\ E_D &= 15 \text{ kip}\end{aligned}$$

JOINT E:



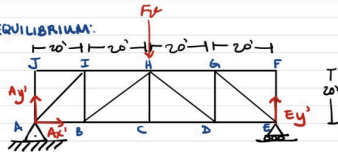
$$\begin{aligned}\sum F_x &= -E_D - E_G\left(\frac{1}{\sqrt{2}}\right) = 0 \\ E_G &= (-15)\left(\frac{1}{\sqrt{2}}\right) \\ E_G &= -21.21 \text{ kip} \quad (\text{compression}) \\ \sum F_y &= E_F + E_G\left(\frac{1}{\sqrt{2}}\right) = 0 \\ E_F &= 15 + (-21.21)\left(\frac{1}{\sqrt{2}}\right) \\ E_F &= 0 \text{ kip} \quad (\text{zero force member})\end{aligned}$$

JOINT G:



$$\begin{aligned}\sum F_x &= G_F + E_G\left(\frac{1}{\sqrt{2}}\right) - H_G = 0 \\ G_F &= -E_G\left(\frac{1}{\sqrt{2}}\right) + H_G \\ G_F &= (-21.21 \text{ kip})\left(\frac{1}{\sqrt{2}}\right) + 15 \text{ kip} \\ G_F &= 0 \text{ kip} \quad (\text{zero force member}) \\ \sum F_y &= -G_D - G_E\left(\frac{1}{\sqrt{2}}\right) = 0 \\ -(-21.21 \text{ kip})\left(\frac{1}{\sqrt{2}}\right) &= G_D \\ 15 \text{ kip} &= G_D\end{aligned}$$

VIRTUAL EQUILIBRIUM:



$$\begin{aligned}\sum F_x &= A_x' = 0 \text{ kip} \\ \sum F_y &= A_y' + E_y' - F_x = 0\end{aligned}$$

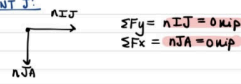
$$\begin{aligned}+\circlearrowleft \sum M_A &= E_y(80 \text{ ft}) - F_x(40 \text{ ft}) = 0 \\ E_y &= \frac{F_x(40 \text{ ft})}{80 \text{ ft}} \\ E_y &= 0.5 \text{ kip}\end{aligned}$$

$$\begin{aligned}A_y' &= F_x - E_y' \\ A_y' &= 1 - 0.5 \text{ kip} \\ A_y' &= 0.5 \text{ kip}\end{aligned}$$

F_v = virtual load
 $A_y = 30.5 \text{ kip}$
 $E_y = 0.5 \text{ kip}$
 $A_x = 0 \text{ kip}$

METHOD OF JOINTS:

JOINT J:



$$\sum F_y = nJI = 0 \text{ kip}$$

$$\sum F_x = nJA = 0 \text{ kip}$$

JOINT A:



$$\sum F_y = nIA \left(\frac{30}{50} \right) + A_y + nJA = 0$$

$$\left(\frac{3}{5} \right) nIA = -A_y$$

$$nIA = (75)(-0.5 \text{ kip})$$

$$nIA = -0.707 \text{ kip}$$

(compression)

$$\sum F_x = nBA + nIA \left(\frac{4}{5} \right) = 0$$

$$nBA = -nIA \left(\frac{4}{5} \right)$$

$$nBA = -(-0.707 \text{ kip}) \left(\frac{4}{5} \right)$$

$$nBA = 0.5 \text{ kip}$$

(tension)

JOINT I:



$$\sum F_x = nIH - nIJ - nIA \left(\frac{4}{5} \right) = 0$$

$$nIH = nIA \left(\frac{4}{5} \right)$$

$$nIH = (-0.707 \text{ kip}) \left(\frac{4}{5} \right)$$

$$nIH = -0.5 \text{ kip}$$

(compression)

$$\sum F_y = -nIJ - nIB \left(\frac{3}{5} \right) = 0$$

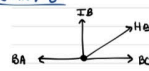
$$-nIB \left(\frac{3}{5} \right) = nIJ$$

$$-(-0.707 \text{ kip}) \left(\frac{3}{5} \right) = nIB$$

$$0.5 \text{ kip} = nIB$$

(tension)

JOINT B:



$$\sum F_y = nIB + nHB \left(\frac{3}{5} \right) = 0$$

$$nHB = (-nIB) \left(\frac{5}{3} \right)$$

$$nHB = (-0.5 \text{ kip}) \left(\frac{5}{3} \right)$$

$$nHB = -0.707 \text{ kip}$$

(compression)

$$\sum F_x = -nBA - nBC + nHB \left(\frac{4}{5} \right) = 0$$

$$nBC = nBA - nHB \left(\frac{4}{5} \right)$$

$$nBC = (0.5 \text{ kip}) - (-0.707 \text{ kip}) \left(\frac{4}{5} \right)$$

$$nBC = 1 \text{ kip}$$

(tension)

JOINT C:



$$\sum F_x = nCD - nBC = 0$$

$$nCD = nBC$$

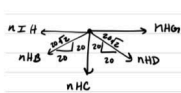
$$nCD = 1 \text{ kip}$$

(tension)

$$\sum F_y = nHC = 0 \text{ kip}$$

(zero member force)

JOINT H:



$$\sum F_y = -nHD \left(\frac{3}{5} \right) - nHC - nHB \left(\frac{3}{5} \right) = 0$$

$$nHD = \left[-nHC - nHB \left(\frac{3}{5} \right) \right] \left(\frac{5}{3} \right)$$

$$nHD = nHB$$

$$nHD = -0.707 \text{ kip}$$

(compression)

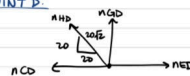
$$\sum F_x = nHI - nIH - nHD \left(\frac{4}{5} \right) - nHB \left(\frac{4}{5} \right) = 0$$

$$nHI = nIH - nHD \left(\frac{4}{5} \right) + nHB \left(\frac{4}{5} \right)$$

$$nHI = -0.5 \text{ kip}$$

(compression)

JOINT D:



$$\sum F_x = nED - nCD - nHD \left(\frac{4}{5} \right) = 0$$

$$nED = nCD + nHD \left(\frac{4}{5} \right)$$

$$nED = 0.5 \text{ kip}$$

(tension)

$$\sum F_y = nAD + nHD \left(\frac{3}{5} \right) = 0$$

$$nAD = -nHD \left(\frac{5}{3} \right)$$

$$nAD = 0.5 \text{ kip}$$

(tension)

JOINT E:



$$F_x = -nED - nEB \left(\frac{4}{5} \right) = 0$$

$$-nEB \left(\frac{4}{5} \right) = nED$$

$$-0.707 \text{ kip} = nEB$$

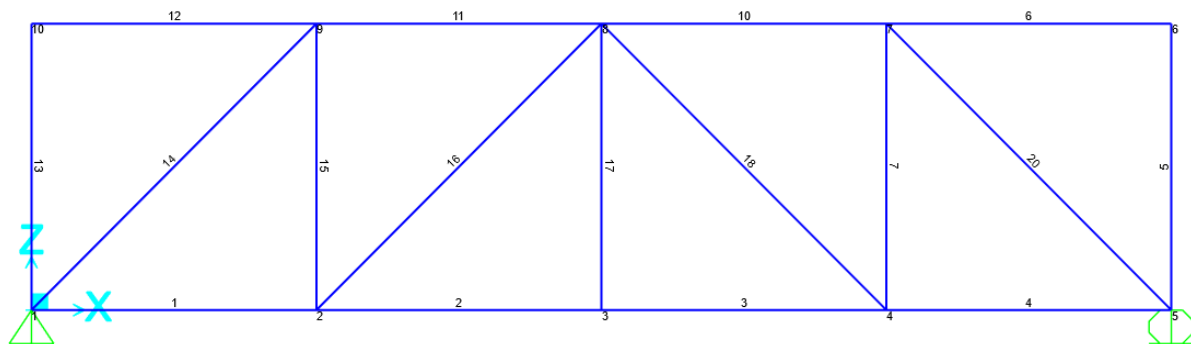
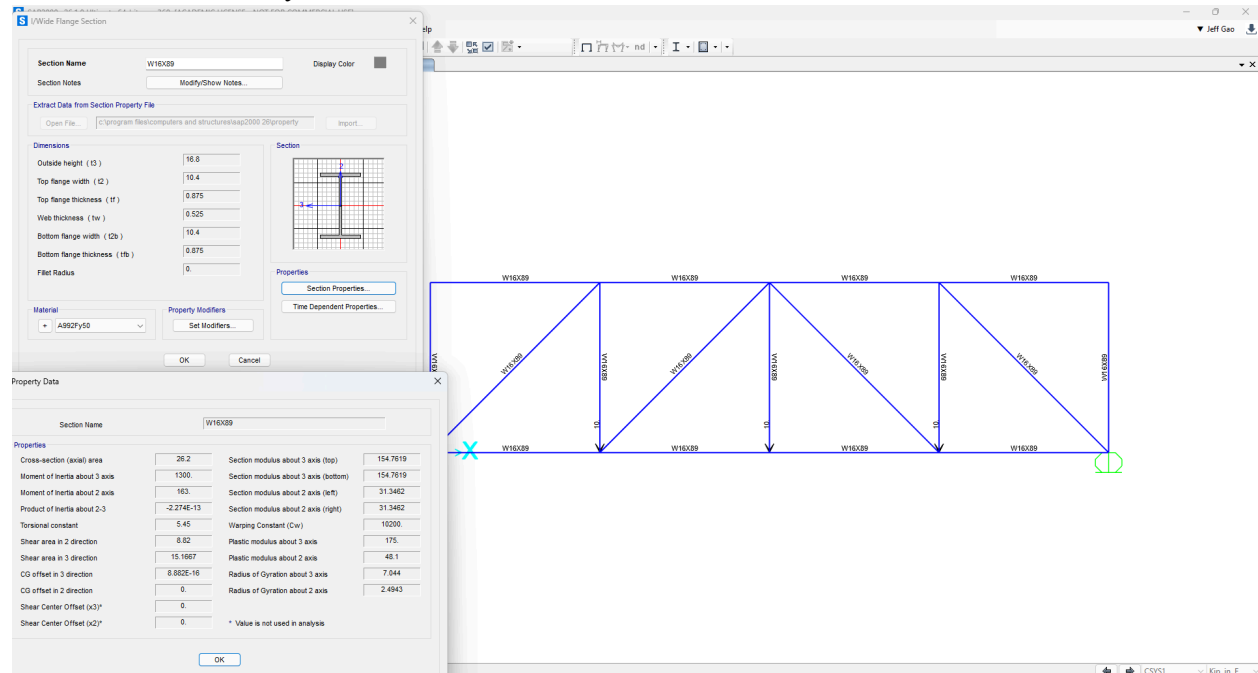
(compression)

$$\sum F_y = nEF = 0 \text{ kip} \text{ (zero force member)}$$

$$\sum F_x = nEB = 0 \text{ kip} \text{ (zero force member)}$$

Truss Analysis with SAP2000

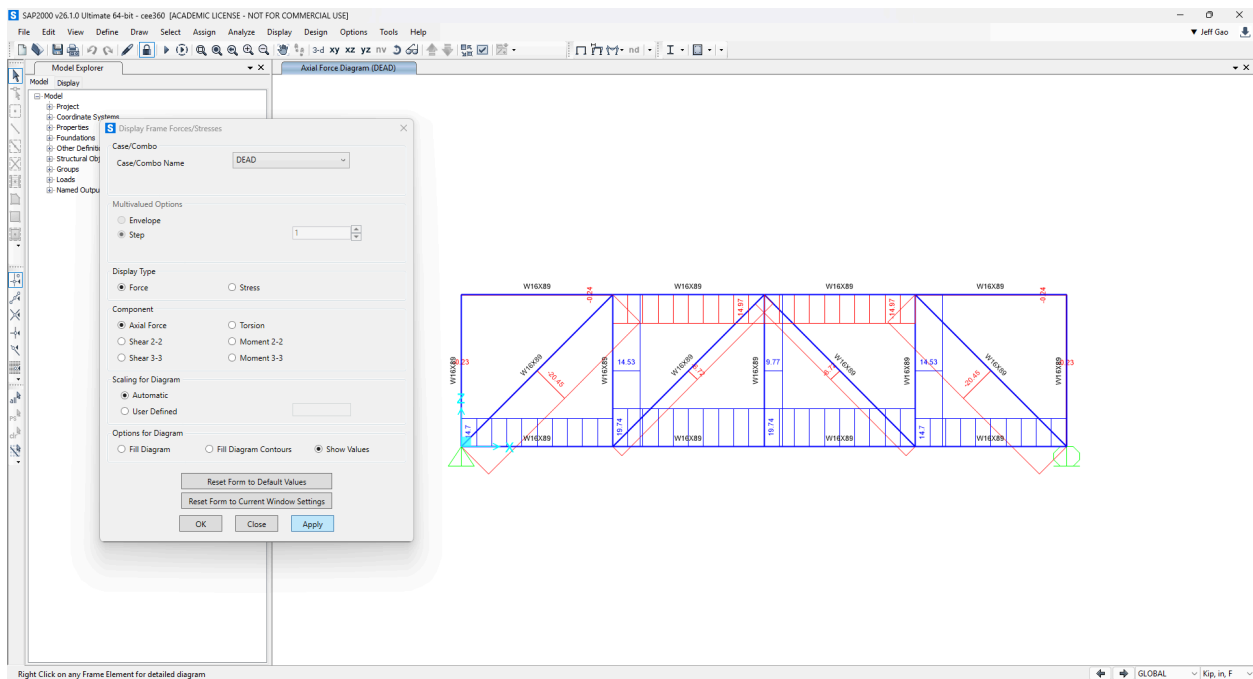
3. The below screenshot shows the geometry of the bridge modeled in SAP2000 with the required material parameters. The other attached screenshot shows the numbering system used in the SAP2000 Analysis

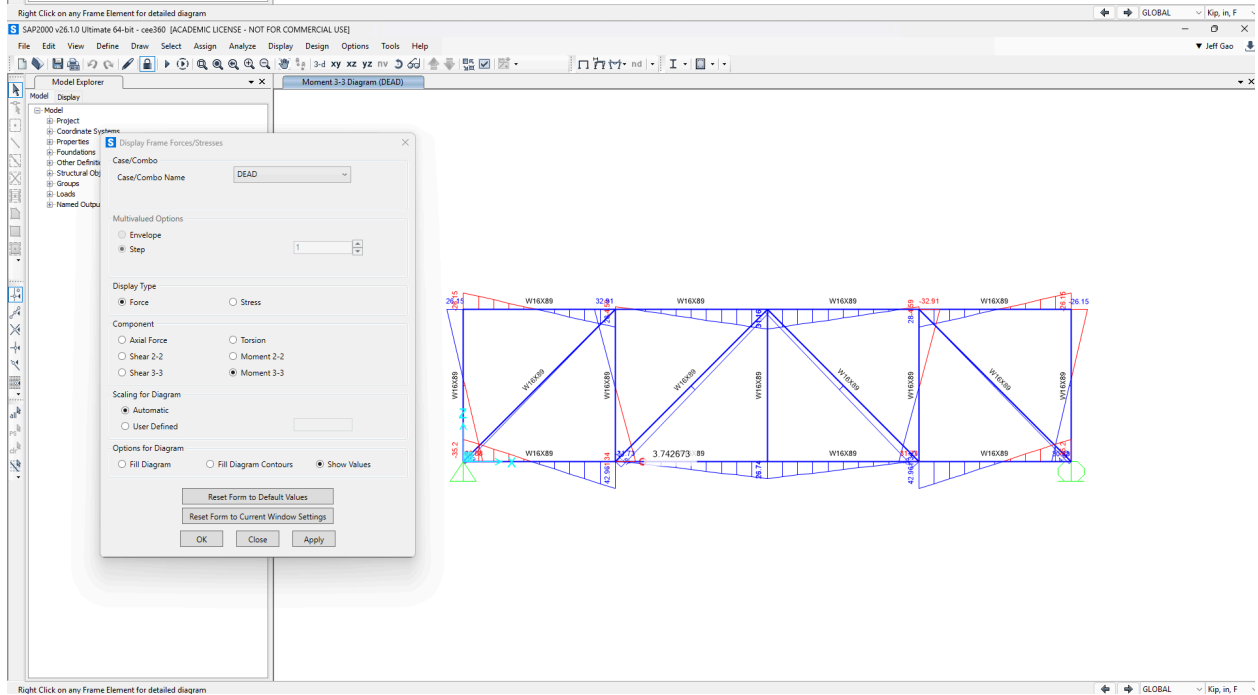
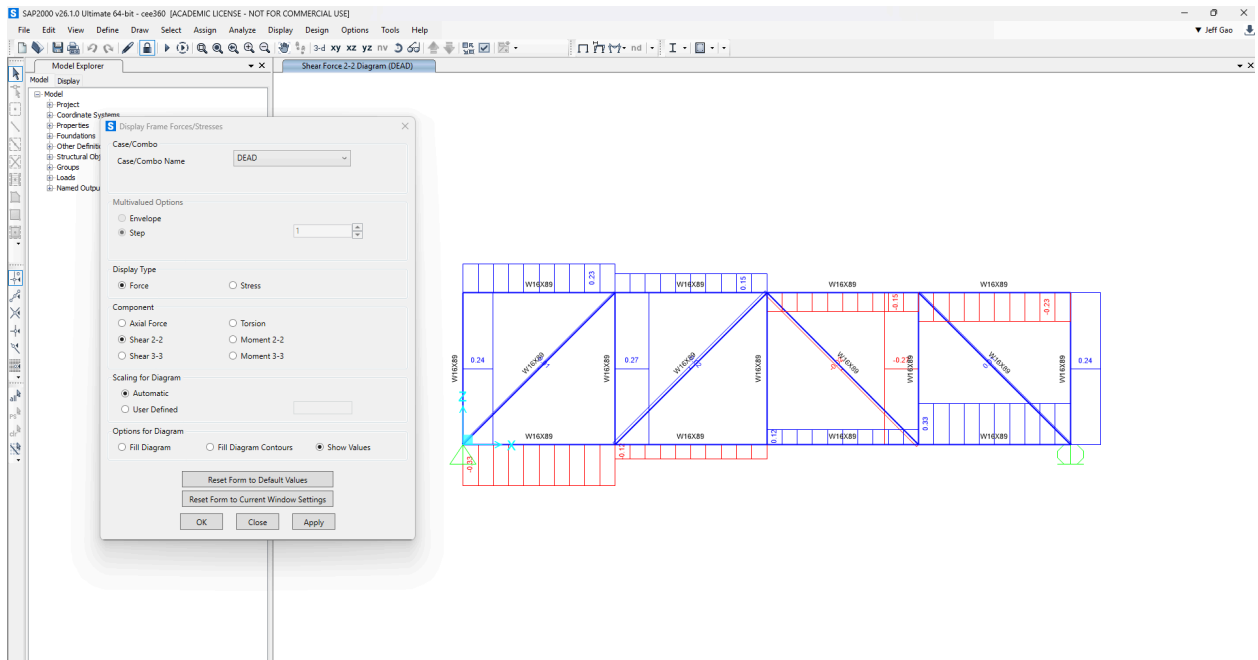


- After applying the external loads and running the simulation we calculated the following member loads. “P” is the axial load. “V2” is the shear force normal to the axis of the members in the XZ plane. “M3” is the moment at each member along the Y axis in and out of the page. All forces/moments are in units of kips and kip-inch.

Table: Element Forces - Frames

Frame	P	V2	M3	FrameElem	Frame	P	V2	M3	FrameElem
	Kip	Kip	Kip-in						
1	14.703	-0.326	-35.204	1-1	11	-14.972	0.149	31.161	11-1
2	19.736	-0.117	-1.343	2-1	12	-0.238	0.226	27.999	12-1
3	19.736	0.117	26.74	3-1	13	-0.226	0.238	26.146	13-1
4	14.703	0.326	42.963	4-1	14	-20.445	0.012	4.328	14-1
5	-0.226	0.238	30.876	5-1	15	14.525	0.269	32.911	15-1
6	-0.238	-0.226	-26.146	6-1	16	-6.716	0.021	12.571	16-1
7	14.525	-0.269	-32.911	7-1	17	9.766	2.22E-16	2.842E-14	17-1
10	-14.972	-0.149	-4.586	10-1	18	-6.716	-0.021	5.387	18-1
					20	-20.445	0.012	4.328	20-1





5. The below screenshots include the overall deformed shape of the structure after the load with the displacement at the center node of the bridge. The table includes the displacements of all the nodes where “U1” corresponds to the displacement in the X axis and “U3” corresponds to the Z axis. “U2” corresponds to the Y axis which isn’t modeled in this scenario.

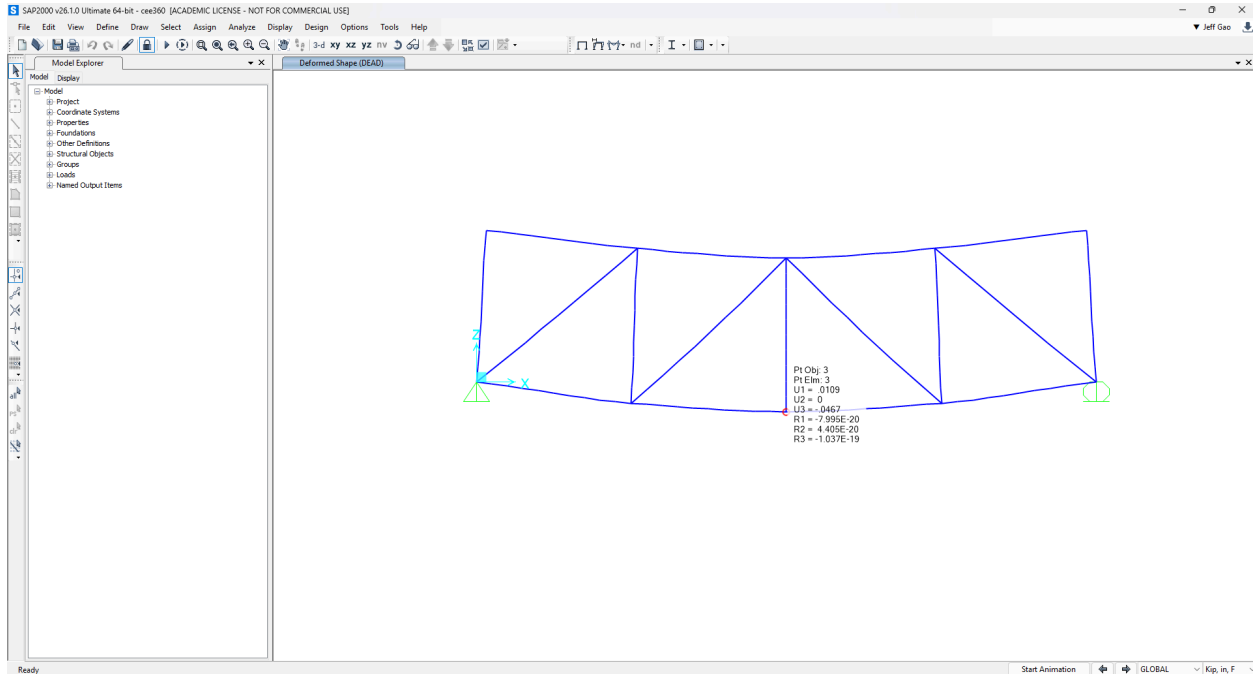


Table: Joint Displacements

Joint	U1	U2	U3	R1	R2	R3
	in	in	in	Radians	Radians	Radians
1	0	0	0	-2.809E-19	0.000106	-6.722E-20
2	0.004644	-3.639E-18	-0.033112	-1.13E-19	8.1E-05	-5.572E-20
3	0.010878	-1.794E-17	-0.046674	-7.995E-20	4.405E-20	-1.037E-19
4	0.017113	-5.188E-17	-0.033112	-1.838E-19	-8.1E-05	-1.324E-19
5	0.021757	-9.481E-17	0	-2.359E-19	-0.000106	-1.171E-19
6	0.006074	-2.339E-17	-7.1E-05	-2.702E-19	-9E-05	-7.703E-20
7	0.006149	-1.47E-17	-0.028524	-2.24E-19	-8.5E-05	-7.079E-20
8	0.010878	1.275E-19	-0.043589	-6.685E-20	2.202E-20	-1.006E-19
9	0.015608	3.517E-17	-0.028524	-1.467E-19	8.5E-05	-1.296E-19
10	0.015683	7.829E-17	-7.1E-05	-2.904E-19	9E-05	-1.464E-19

Matrix Analysis with Python

For the matrix analysis, the members were assigned numbers and degrees of freedom as follows

[insert drawn diagram here]

6. The displacement transformation matrix for each member is found using λ_x and λ_y . Horizontal members have no λ_y component, and vertical members have no λ_x component. The diagonal members have $\sqrt{2}$ multipliers where necessary. Code is shown in the appendix.
7. To construct the global stiffness matrix, the individual stiffness matrices of each member were created using the general structure derived in lecture and λ_x and λ_y . Next, an empty matrix of the dimensions $N \times N$ was created (N being the DOF). Using nested for loops, each member's matrix was added into the correct spot in the global stiffness matrix. Code is shown in the appendix.
8. Once the global matrix was completed, the forces and displacements can be found. First, the force vector is defined. Once boundary conditions are specified, and the free displacements are solved for, the displacement vector can be created. This gives the answer to question 9. Finally, the forces can be printed out by multiplying the displacements by EA/L . This prints all the forces, which were identical to the ones found in Q2 and with SAP. Code is shown in the appendix.
9. The displacements were already solved for and printed out with the code used to answer Q8. The corresponding displacement value for the midspan deflection is "DOF 6".

Appendix:

Code:

```
import numpy as np
import sympy as sp

L = 20 #ft
L = 20*12
E = 29000 #ksi
A = 26.2 #in^2

r2 = np.sqrt(2)
```

Question 6:

```
#Question 6      see lec 29 slide 11, that matrix can be applied to each
member

mem_1_T = np.array([[0,-L/L,0,0],
                    [0,0,0,-L/L]])

mem_2_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_3_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_4_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_5_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_6_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_7_T = np.array([[1/r2,1/r2,0,0],          #diagonal
                    [0,0,1/r2,1/r2]])

mem_8_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_9_T = np.array([[L/L,0,0,0],
```

```

[0,0,L/L,0]])

mem_10_T = np.array([[L/L,0,0,0],
                    [0,0,L/L,0]])

mem_11_T = np.array([[0,-L/L,0,0],
                    [0,0,0,-L/L]])

mem_12_T = np.array([[1/r2,1/r2,0,0],          #diagonal
                    [0,0,1/r2,1/r2]])

mem_13_T = np.array([[0,-L/L,0,0],
                    [0,0,0,-L/L]])

mem_14_T = np.array([[1/r2,-1/r2,0,0],         #diagonal
                    [0,0,1/r2,-1/r2]])

mem_15_T = np.array([[0,-L/L,0,0],
                    [0,0,0,-L/L]])

mem_16_T = np.array([[1/r2,-1/r2,0,0],         #diagonal
                    [0,0,1/r2,-1/r2]])

mem_17_T = np.array([[0,-L/L,0,0],
                    [0,0,0,-L/L]])

```

Question 6 Output:

```
... [[ 0. -1.  0.  0.]
      [ 0.  0.  0. -1.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[0.70710678 0.70710678 0.          0.          ]
        [0.          0.          0.70710678 0.70710678]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[1.  0.  0.  0.]
        [0.  0.  1.  0.]]
      [[ 0. -1.  0.  0.]
        [ 0.  0.  0. -1.]]
      [[0.70710678 0.70710678 0.          0.          ]
        [0.          0.          0.70710678 0.70710678]]
      [[ 0. -1.  0.  0.]
        [ 0.  0.  0. -1.]]
      [[ 0.70710678 -0.70710678 0.          0.          ]
        [ 0.          0.          0.70710678 -0.70710678]]
      [[ 0. -1.  0.  0.]
        [ 0.  0.  0. -1.]]
      [[ 0.70710678 -0.70710678 0.          0.          ]
        [ 0.          0.          0.70710678 -0.70710678]]
      [[ 0. -1.  0.  0.]
        [ 0.  0.  0. -1.]]
```

Question 7:

```
#matrix is same lambdas are different, global can be found in lec 30
N = 20 #number of DOF
K = np.zeros((N,N))
guide = np.array([0, 1, 2, 3])

k_mem_1 = ((E*A)/L) * np.array([[0, 0, 0, 0],
                                  [0, 1, 0, -1],
                                  [0, 0, 0, 0],
                                  [0, -1, 0, 1]])

k_mem_2 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                  [0, 0, 0, 0],
                                  [-1, 0, 1, 0],
                                  [0, 0, 0, 0]])

k_mem_3 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                  [0, 0, 0, 0],
                                  [-1, 0, 1, 0],
                                  [0, 0, 0, 0]])

k_mem_4 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                  [0, 0, 0, 0],
                                  [-1, 0, 1, 0],
                                  [0, 0, 0, 0]])

k_mem_5 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                  [0, 0, 0, 0],
                                  [-1, 0, 1, 0],
                                  [0, 0, 0, 0]])

k_mem_6 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                  [0, 0, 0, 0],
                                  [-1, 0, 1, 0],
                                  [0, 0, 0, 0]])

k_mem_7 = ((E*A)/L) * np.array([[1/2, 1/2, -1/2, -1/2],      #diagonal
                                  [1/2, 1/2, -1/2, -1/2],
```

```

        [-1/2, -1/2, 1/2, 1/2],
        [-1/2, -1/2, 1/2, 1/2]])

k_mem_8 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                [0, 0, 0, 0],
                                [-1, 0, 1, 0],
                                [0, 0, 0, 0]])

k_mem_9 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                [0, 0, 0, 0],
                                [-1, 0, 1, 0],
                                [0, 0, 0, 0]])

k_mem_10 = ((E*A)/L) * np.array([[1, 0, -1, 0],
                                [0, 0, 0, 0],
                                [-1, 0, 1, 0],
                                [0, 0, 0, 0]])

k_mem_11 = ((E*A)/L) * np.array([[0, 0, 0, 0],
                                [0, 1, 0, -1],
                                [0, 0, 0, 0],
                                [0, -1, 0, 1]])

k_mem_12 = ((E*A)/L) * np.array([[1/2, 1/2, -1/2, -1/2],      #diagonal
                                [1/2, 1/2, -1/2, -1/2],
                                [-1/2, -1/2, 1/2, 1/2],
                                [-1/2, -1/2, 1/2, 1/2]])

k_mem_13 = ((E*A)/L) * np.array([[0, 0, 0, 0],
                                [0, 1, 0, -1],
                                [0, 0, 0, 0],
                                [0, -1, 0, 1]])

k_mem_14 = ((E*A)/L) * np.array([[1/2, -1/2, -1/2, 1/2],      #diagonal
                                [-1/2, 1/2, 1/2, -1/2],
                                [-1/2, 1/2, 1/2, -1/2],
                                [1/2, -1/2, -1/2, 1/2]])

k_mem_15 = ((E*A)/L) * np.array([[0, 0, 0, 0],
                                [0, 1, 0, -1],

```

```

        [0, 0, 0, 0],
        [0, -1, 0, 1]])

k_mem_16 = ((E*A)/L) * np.array([[1/2, -1/2, -1/2, 1/2],      #diagonal
                                  [-1/2, 1/2, 1/2, -1/2],
                                  [-1/2, 1/2, 1/2, -1/2],
                                  [1/2, -1/2, -1/2, 1/2]])

k_mem_17 = ((E*A)/L) * np.array([[0, 0, 0, 0],
                                   [0, 1, 0, -1],
                                   [0, 0, 0, 0],
                                   [0, -1, 0, 1]])

guideK1 = np.array([19-1, 20-1, 1-1, 2-1])
for i in guide:
    for j in guide:
        K[guideK1[i],guideK1[j]] = k_mem_1[i,j] + K[guideK1[i],guideK1[j]]

guideK2 = np.array([1-1, 2-1, 3-1, 4-1])
for i in guide:
    for j in guide:
        K[guideK2[i],guideK2[j]] = k_mem_2[i,j] + K[guideK2[i],guideK2[j]]

guideK3 = np.array([3-1, 4-1, 5-1, 6-1])
for i in guide:
    for j in guide:
        K[guideK3[i],guideK3[j]] = k_mem_3[i,j] + K[guideK3[i],guideK3[j]]

guideK4 = np.array([5-1, 6-1, 7-1, 8-1])
for i in guide:
    for j in guide:
        K[guideK4[i],guideK4[j]] = k_mem_4[i,j] + K[guideK4[i],guideK4[j]]

guideK5 = np.array([7-1, 8-1, 9-1, 10-1])
for i in guide:
    for j in guide:
        K[guideK5[i],guideK5[j]] = k_mem_5[i,j] + K[guideK5[i],guideK5[j]]

guideK6 = np.array([19-1, 20-1, 17-1, 18-1])
for i in guide:

```



```

    for j in guide:
        K[guideK6[i],guideK6[j]] = k_mem_6[i,j] + K[guideK6[i],guideK6[j]]

guideK7 = np.array([1-1, 2-1, 17-1, 18-1])
for i in guide:
    for j in guide:
        K[guideK7[i],guideK7[j]] = k_mem_7[i,j] + K[guideK7[i],guideK7[j]]

guideK8 = np.array([17-1, 18-1, 15-1, 16-1])
for i in guide:
    for j in guide:
        K[guideK8[i],guideK8[j]] = k_mem_8[i,j] + K[guideK8[i],guideK8[j]]

guideK9 = np.array([15-1, 16-1, 13-1, 14-1])
for i in guide:
    for j in guide:
        K[guideK9[i],guideK9[j]] = k_mem_9[i,j] + K[guideK9[i],guideK9[j]]

guideK10 = np.array([13-1, 14-1, 11-1, 12-1])
for i in guide:
    for j in guide:
        K[guideK10[i],guideK10[j]] = k_mem_10[i,j] +
K[guideK10[i],guideK10[j]]

guideK11 = np.array([17-1, 18-1, 3-1, 4-1])
for i in guide:
    for j in guide:
        K[guideK11[i],guideK11[j]] = k_mem_11[i,j] +
K[guideK11[i],guideK11[j]]

guideK12 = np.array([3-1, 4-1, 15-1, 16-1])
for i in guide:
    for j in guide:
        K[guideK12[i],guideK12[j]] = k_mem_12[i,j] +
K[guideK12[i],guideK12[j]]

guideK13 = np.array([15-1, 16-1, 5-1, 6-1])
for i in guide:
    for j in guide:

```

```

        K[guideK13[i],guideK13[j]] = k_mem_13[i,j] +
K[guideK13[i],guideK13[j]]

guideK14 = np.array([15-1, 16-1, 7-1, 8-1])
for i in guide:
    for j in guide:
        K[guideK14[i],guideK14[j]] = k_mem_14[i,j] +
K[guideK14[i],guideK14[j]]

guideK15 = np.array([13-1, 14-1, 7-1, 8-1])
for i in guide:
    for j in guide:
        K[guideK15[i],guideK15[j]] = k_mem_15[i,j] +
K[guideK15[i],guideK15[j]]

guideK16 = np.array([13-1, 14-1, 9-1, 10-1])
for i in guide:
    for j in guide:
        K[guideK16[i],guideK16[j]] = k_mem_16[i,j] +
K[guideK16[i],guideK16[j]]

guideK17 = np.array([11-1, 12-1, 9-1, 10-1])
for i in guide:
    for j in guide:
        K[guideK17[i],guideK17[j]] = k_mem_17[i,j] +
K[guideK17[i],guideK17[j]]

```

Question 7 Output:

```

... [[ 4748.75      1582.91666667 -3165.83333333      0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      -1582.91666667 -1582.91666667      0.          0.          ]
 [ 1582.91666667  4748.75      0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      -1582.91666667 -1582.91666667      0.          -3165.83333333]
 [-3165.83333333      0.          7914.58333333  1582.91666667
 -3165.83333333      0.          0.          0.
      0.          0.          0.          0.
      0.          0.          -1582.91666667 -1582.91666667
      0.          0.          0.          0.          ]
 [      0.          0.          1582.91666667  4748.75
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          -1582.91666667 -1582.91666667
      0.          -3165.83333333      0.          0.          ]
 [      0.          0.          -3165.83333333      0.
 6331.66666667      0.          -3165.83333333      0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          -3165.83333333
      0.          0.          0.          0.          ]
 [      0.          0.          0.          0.
 -3165.83333333      0.          7914.58333333 -1582.91666667
 -3165.83333333      0.          0.          0.
      0.          0.          -1582.91666667  1582.91666667
      0.          0.          0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          -1582.91666667  4748.75
      0.          0.          0.          0.
      0.          -3165.83333333  1582.91666667 -1582.91666667
      0.          0.          0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          -3165.83333333      0.
 4748.75      -1582.91666667      0.          0.
 -1582.91666667  1582.91666667      0.          0.
      0.          0.          0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      -1582.91666667  4748.75      0.          -3165.83333333
      1582.91666667 -1582.91666667      0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          3165.83333333      0.
 -3165.83333333      0.          0.          0.          ]
 [      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          0.          0.          0.
      0.          -3165.83333333      0.          3165.83333333

```

```

0.      0.      0.      0.      ]
[  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      -3165.83333333  0.      3165.83333333
  0.      0.      0.      0.
  0.      0.      0.      0.      ]
[  0.      0.      0.      0.
  0.      0.      0.      0.
-1582.91666667  1582.91666667 -3165.83333333  0.
 7914.58333333 -1582.91666667 -3165.83333333  0.
  0.      0.      0.      0.      ]
[  0.      0.      0.      0.
  0.      0.      0.      -3165.83333333
 1582.91666667 -1582.91666667  0.      0.
-1582.91666667  4748.75  0.      0.
  0.      0.      0.      0.      ]
[  0.      0.      -1582.91666667 -1582.91666667
  0.      0.      -1582.91666667  1582.91666667
  0.      0.      0.      0.
-3165.83333333  0.      9497.5  0.
-3165.83333333  0.      0.      0.      ]
[  0.      0.      -1582.91666667 -1582.91666667
  0.      -3165.83333333  1582.91666667 -1582.91666667
  0.      0.      0.      0.
  0.      0.      0.      6331.66666667
  0.      0.      0.      0.      ]
[-1582.91666667 -1582.91666667  0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      -3165.83333333  0.
 7914.58333333  1582.91666667 -3165.83333333  0.      ]
[-1582.91666667 -1582.91666667  0.      -3165.83333333
  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
 1582.91666667  4748.75  0.      0.      ]
[  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
-3165.83333333  0.      3165.83333333  0.      ]
[  0.      -3165.83333333  0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      0.      0.
  0.      0.      0.      3165.83333333]]

```

Question 8 & 9:

```
# Define the force vector

F = np.zeros(N)

F[3] = -10    # joint 2
F[5] = -10    # joint 3
F[7] = -10    # joint 4

# boundary conditions

fixed_dofs = [0, 1, 9]
free_dofs = [i for i in range(N) if i not in fixed_dofs]

# stiffness matrix
K_ff = K[np.ix_(free_dofs, free_dofs)]
F_f = F[free_dofs]

# free displacements
U_free = np.linalg.solve(K_ff, F_f)

# displacement vector
U = np.zeros(N)
for i, dof in enumerate(free_dofs):
    U[dof] = U_free[i]

print("Nodal displacements (inches):")
for i in range(N):
    print(f" DOF {i+1}: {U[i]:.6f} in")

# Calculating member forces:
member_guides = [
    guideK1, guideK2, guideK3, guideK4, guideK5,
    guideK6, guideK7, guideK8, guideK9, guideK10,
    guideK11, guideK12, guideK13, guideK14, guideK15,
```

```

        guideK16, guideK17
    ]
member_T = [
    mem_1_T, mem_2_T, mem_3_T, mem_4_T, mem_5_T,
    mem_6_T, mem_7_T, mem_8_T, mem_9_T, mem_10_T,
    mem_11_T, mem_12_T, mem_13_T, mem_14_T, mem_15_T,
    mem_16_T, mem_17_T
]

print("\nMember Forces (kips):")
for m in range(17):
    guide = member_guides[m]
    T = member_T[m]
    u_local = np.array([U[guide[0]], U[guide[1]], U[guide[2]],
U[guide[3]]])
    # Force = (EA/L) * [-1, 1] @ T @ u_local (deformation in local
coords)
    deformation = T @ u_local # [u_local_i, u_local_j]
    force = (E * A / L) * (deformation[1] - deformation[0])
    print(f" Member {m+1}: {force:.2f} kips")

```

Question 8 Output:

• Nodal displacements (inches):

DOF 1: 0.000000 in
DOF 2: 0.000000 in
DOF 3: 0.004738 in
DOF 4: -0.030008 in
DOF 5: 0.011056 in
DOF 6: -0.042643 in
DOF 7: 0.017373 in
DOF 8: -0.030008 in
DOF 9: 0.022111 in
DOF 10: 0.000000 in
DOF 11: 0.006317 in
DOF 12: 0.000000 in
DOF 13: 0.006317 in
DOF 14: -0.025270 in
DOF 15: 0.011056 in
DOF 16: -0.039484 in
DOF 17: 0.015794 in
DOF 18: -0.025270 in
DOF 19: 0.015794 in
DOF 20: 0.000000 in

Member Forces (kips):

Member 1: 0.00 kips
Member 2: 15.00 kips
Member 3: 20.00 kips
Member 4: 20.00 kips
Member 5: 15.00 kips
Member 6: 0.00 kips
Member 7: -21.21 kips
Member 8: -15.00 kips
Member 9: -15.00 kips
Member 10: 0.00 kips
Member 11: 15.00 kips
Member 12: -7.07 kips
Member 13: 10.00 kips
Member 14: -7.07 kips
Member 15: 15.00 kips
Member 16: -21.21 kips
Member 17: 0.00 kips